in partial fulfillment for the award of the Degree

INFORMATION TECHNOLOGY

(AN AUTONOMOUS INSTITUTION) - AFFILIATED TO ANNA UNIVERSITY,

HEAD OF THE DEPARTMENT **SUPERVISOR**

_____ _____

Dr. [HOD Name Placeholder] **Mr. GRANDHI SAI**

Head of Department **Project Supervisor**

Department of Information Technology **Department of Information Technology**

Easwari Engineering College **Easwari Engineering College**

Chennai - 600 089 **Chennai - 600 089**

Internal Examiner

External Examiner

Name : [Internal Examiner Name]

Name : [External Examiner Name]

Designation : [Designation]

Designation : [Designation]

Address : Easwari Engineering College,

Chennai - 600 089

Address : [Institution / University],

Chennai

Date : _____ **Date : _____**

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
CHAPTER 1	**INTRODUCTION**	**1**
1.1	OVERVIEW	1
1.2	PROBLEM DEFINITION	1
1.3	OBJECTIVE	2
1.4	SCOPE OF THE PROJECT	2
1.5	MOTIVATION	3
CHAPTER 2	**RELATED WORK**	**4**
2.1	LITERATURE REVIEW	4
2.2	EXISTING SYSTEM	5

2.3	PROPOSED SYSTEM	5
CHAPTER 3	**SYSTEM ANALYSIS**	**7**
3.1	SYSTEM REQUIREMENTS	7
3.2	SOFTWARE AND HARDWARE REQUIREMENTS	8
CHAPTER 4	**SYSTEM DESIGN**	**10**
4.1	SYSTEM ARCHITECTURE	10
4.2	MODULE DESCRIPTION	11
4.4	CLASS DIAGRAM	12
4.5	SEQUENCE DIAGRAM	13
CHAPTER 5	**IMPLEMENTATION**	**15**
5.1	IMPLEMENTATION OVERVIEW	15
5.2	KEY FUNCTIONS (CODE EXPLANATION)	16

CHAPTER 6	**CONCLUSION & FUTURE WORK**	**20**
6.1	CONCLUSION	20
6.2	FUTURE ENHANCEMENTS	21
	APPENDIX - SOURCE CODE	**22**
	SCREENSHOTS OF THE PROJECT	**31**
	REFERENCES	**35**

Hardware Resource	Minimum Specification	Recommended Specification
Processor (CPU)	Dual-Core 2.0 GHz x86/x64	Quad-Core Intel Core i5 / AMD Ryzen 5 or higher
System Memory (RAM)	4 GB DDR4	8 GB to 16 GB DDR4/DDR5
Storage Capacity	2 GB available disk space	10 GB Solid State Drive (SSD) space
Display Resolution	1280 x 720 (HD)	1920 x 1080 (Full HD, 1080p)
Data Visualization	Recharts	v2.15.1 (SVG-based reactive charts)
Backend Runtime	Node.js	v18.0.0 or higher (v20+ LTS verified)
API Framework	Express.js	v4.21.2 (RESTful middleware stack)
Database Technology	MongoDB	v7.0+ / In-Memory MongoMemoryServer
Object Data Modeling	Mongoose ODM	v8.9.5 (Schema validation & hooks)
Security & Cryptography	bcryptjs & jsonwebtoken	bcryptjs v2.4.3 / JWT v9.0.2

3. Data Persistence Tier (Database): Implemented using MongoDB and modeled through Mongoose schemas. The system features a resilient hybrid database architecture that automatically connects to local or cloud MongoDB instances on port 27017, seamlessly falling back to an embedded in-memory MongoDB server (MongoMemoryServer) for zero-setup local deployment.

• Transaction Model: Records the immutable banking ledger entry containing transactionId, generated referenceNumber (Bank UTR), exact settled amount, and settlement timestamp.

9. The React client updates its local state cache, displays an interactive toast notification, and renders the verification badge in real time.

• File Path: server/services/verificationService.js

• Function Name: verifyInvoice(invoiceData)

```
const verifyInvoice = async (invoiceData) => {
  const checks = {
    vendorActive:     { passed: false, score: 20, maxScore: 20, message: '' },
    formatValid:      { passed: false, score: 15, maxScore: 15, message: '' },
    datesConsistent:  { passed: false, score: 20, maxScore: 20, message: '' },
    amountPositive:   { passed: false, score: 15, maxScore: 15, message: '' },
    lineMathCorrect:  { passed: false, score: 15, maxScore: 15, message: '' },
    gstValid:         { passed: false, score: 15, maxScore: 15, message: '' },
  };

  // Rule 1: Vendor Active in Registry
  const vendor = await Vendor.findById(invoiceData.vendor);
  if (vendor && vendor.status === 'ACTIVE') {
    checks.vendorActive.passed = true;
  }
  // Rule 2: Format Match (INV-YYYY-XXX)
  const formatRegex = /^INV-\d{4}-\d{3,6}$/i;
  checks.formatValid.passed = formatRegex.test(invoiceData.invoiceNumber);

  // Rule 3: Date Chronology
  const invDate = new Date(invoiceData.invoiceDate);
  const dueDate = new Date(invoiceData.dueDate);
  checks.datesConsistent.passed = dueDate >= invDate;

  // Rule 4 & 5: Amount Math & Line Items
  const calculatedSubtotal = invoiceData.lineItems.reduce((acc, item) =>
    acc + (item.quantity * item.unitPrice), 0);
  checks.lineMathCorrect.passed = Math.abs(calculatedSubtotal -
invoiceData.subtotal) < 1.0;

  // Rule 6: 18% GST Calculation
  const expectedTax = calculatedSubtotal * 0.18;
  checks.gstValid.passed = Math.abs(expectedTax - invoiceData.taxAmount) < 1.0;

  const totalScore = Object.values(checks).reduce((sum, c) => sum + (c.passed ?
c.score : 0), 0);
  return { checks, score: totalScore, status: totalScore >= 80 ? 'VERIFIED' :
'NEEDS REVIEW' };
};
```

• File Path: server/services/duplicateService.js

• Function Name: checkDuplicateInvoice(invoiceData)

```
const levenshteinDistance = (str1, str2) => {
```

```
  const m = str1.length, n = str2.length;
  const dp = Array.from({ length: m + 1 }, () => Array(n + 1).fill(0));
  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      dp[i][j] = str1[i - 1] === str2[j - 1]
        ? dp[i - 1][j - 1]
        : 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
    }
  }
  return dp[m][n];
};

const checkDuplicateInvoice = async (invoiceData) => {
  const candidates = await Invoice.find({
    vendor: invoiceData.vendor,
    _id: { $ne: invoiceData._id }
  });
  let highestConfidence = 0;
  let matchingInvoice = null;

  for (const c of candidates) {
    const dist = levenshteinDistance(invoiceData.invoiceNumber.toUpperCase(),
c.invoiceNumber.toUpperCase());
    const maxLen = Math.max(invoiceData.invoiceNumber.length,
c.invoiceNumber.length);
    const numSimilarity = ((maxLen - dist) / maxLen) * 40; // 40% weight
    const amtMatch = Math.abs(invoiceData.totalAmount - c.totalAmount) < 0.01 ?
25 : 0; // 25% weight
    const vendorMatch = 20; // 20% weight
    const dateDiffDays = Math.abs(new Date(invoiceData.invoiceDate) - new
Date(c.invoiceDate)) / 86400000;
    const dateMatch = dateDiffDays < 7 ? 15 : 0; // 15% weight

    const confidence = numSimilarity + amtMatch + vendorMatch + dateMatch;
    if (confidence > highestConfidence) {
      highestConfidence = confidence;
      matchingInvoice = c;
    }
  }
  return { isDuplicate: highestConfidence >= 75, confidence: highestConfidence,
duplicateOf: matchingInvoice };
};
```

• File Path: server/services/paymentService.js

• Function Name: processPayment(paymentId, user)

```
const processPayment = async (paymentId, user) => {
  const payment = await Payment.findById(paymentId).populate('invoice vendor');
  if (!payment) throw new Error('Payment record not found');
  if (payment.status === 'COMPLETED') throw new Error('Payment already
settled');

  // Generate authentic Indian Bank UTR reference code
  const utr = `UTR-HDFC${new Date().getFullYear()}${Math.floor(100000 +
Math.random() * 900000)}`;

  // Create permanent Transaction ledger entry
  const transaction = await Transaction.create({
    transactionId: `TXN-${Date.now()}`,
    payment: payment._id,
    invoice: payment.invoice._id,
    vendor: payment.vendor._id,
    amount: payment.amount,
    paymentMethod: payment.paymentMethod || 'NEFT',
    referenceNumber: utr,
    status: 'SUCCESS',
    timestamp: new Date()
  });

  payment.status = 'COMPLETED';
  payment.transaction = transaction._id;
  payment.processedAt = new Date();
  await payment.save();

  // Update parent Invoice status to PAID
  await Invoice.findByIdAndUpdate(payment.invoice._id, {
    status: 'PAID',
    paymentStatus: 'PAID'
  });

  return { payment, transaction, utr };
};
```

• File Path: server/utils/datePriority.js

• Function Name: calculatePriority(dueDate, status)

```
const calculatePriority = (dueDate, status) => {
  if (['PAID', 'REJECTED', 'CANCELLED'].includes(status)) {
    return { priority: 'LOW', isOverdue: false, daysDiff: 0 };
  }
  const today = new Date();
  today.setHours(0, 0, 0, 0);
  const due = new Date(dueDate);
```

```
  due.setHours(0, 0, 0, 0);
  const diffDays = Math.round((due - today) / (1000 * 60 * 60 * 24));

  if (diffDays < 0) {
    return { priority: 'OVERDUE', isOverdue: true, daysDiff: Math.abs(diffDays)
};
  } else if (diffDays <= 3) {
    return { priority: 'URGENT', isOverdue: false, daysDiff };
  } else if (diffDays <= 7) {
    return { priority: 'HIGH', isOverdue: false, daysDiff };
  }
  return { priority: 'NORMAL', isOverdue: false, daysDiff };
};
```

```
INVOXA-PLATFORM/
├── package.json                    # Root orchestrator (concurrently)
├── README.md                       # Platform documentation & quickstart
├── Documentation_Output/           # Generated project reports & diagrams
│   ├── INVOXA_Project_Report.docx # Complete editable Word report
│   ├── INVOXA_Project_Report.pdf  # Paginated PDF document
│   ├── Diagrams/                   # Architecture, Class, Sequence PNGs
│   └── Screenshots/                # Authenticated UI screenshots
│
├── client/                         # React 18 + Vite Frontend Client
│   ├── package.json                # Client dependencies (React, Tailwind,
Recharts)
│   ├── vite.config.js              # Vite build & proxy configuration
│   ├── tailwind.config.js          # Custom Warm Charcoal theme tokens
│   └── src/
│       ├── App.jsx                 # Application routes & ProtectedRoute guards
│       ├── main.jsx                # React DOM entry point
│       ├── components/             # Reusable UI components
│       │   ├── common/             # StatCard, Modal, StatusBadge, Skeleton
│       │   ├── dashboard/          # OverdueBanner, PendingApprovalsWidget
│       │   ├── layout/             # AppLayout, Navbar, Sidebar
│       │   └── ui/                 # WelcomeAnimation, IntroScreen
│       ├── context/                # AuthContext, ThemeContext, ToastContext
│       ├── pages/                  # Full view components
│       │   ├── auth/               # LoginPage.jsx
│       │   ├── dashboard/          # DashboardPage.jsx
│       │   ├── invoices/           # InvoicesPage.jsx (CRUD + Upload)
│       │   ├── verification/       # VerificationPage.jsx
│       │   ├── approvals/          # ApprovalsPage.jsx
│       │   ├── payments/           # PaymentsPage.jsx
│       │   ├── transactions/       # TransactionsPage.jsx
│       │   ├── vendors/            # VendorsPage.jsx
│       │   ├── reports/            # ReportsPage.jsx (Cashflow & Funnels)
│       │   └── activity/           # ActivityLogPage.jsx
│       └── services/               # Centralized api.js HTTP client
│
└── server/                         # Express.js REST API Backend Server
    ├── package.json                # Backend dependencies (Express, Mongoose,
JWT)
    ├── server.js                   # Server entry point & reminder interval
    ├── app.js                      # Express app setup & route mounting
    ├── .env.example                # Environment variable templates
    ├── config/                     # db.js (MongoDB + MongoMemoryServer)
    ├── models/                     # Mongoose schemas (User, Vendor, Invoice,
etc.)
    ├── controllers/                # Route handlers (auth, invoice, report,
etc.)
    ├── routes/                     # API endpoints (/api/auth, /api/invoices,
etc.)
    ├── services/                   # Business logic (verify, duplicate, payment)
```

```
    ├── middleware/               # authMiddleware, roleMiddleware, multer
    ├── utils/                    # apiResponse.js, datePriority.js
    ├── seed/                     # seeder.js (Auto-seed demo data)
    └── uploads/                  # Secure storage for invoice digital files
```

```
# 1. Navigate to the backend server directory
cd server

# 2. Install production dependencies
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer morgan
mongodb-memory-server

# 3. Install developer dependencies for live code reloading
npm install -D nodemon

# 4. Optional: Run database seeder to populate sample enterprise data
npm run seed

# 5. Start development API server on port 5000
npm run dev
```

```
// File: server/config/db.js
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');

let mongodInstance = null;

const connectDB = async () => {
  const uri = process.env.MONGODB_URI ||
'mongodb://127.0.0.1:27017/invoiceflow';
  try {
    // Attempt connecting to the specified URI first with a 2-second timeout
    await mongoose.connect(uri, { serverSelectionTimeoutMS: 2000 });
    console.log(`[Database] Connected to external/local MongoDB at ${uri}`);
  } catch (err) {
    console.warn(`[Database] Could not connect to MongoDB at ${uri}
(${err.message}).`);
    console.log('[Database] Initializing embedded in-memory MongoDB server for
zero-setup execution...');
    try {
      mongodInstance = await MongoMemoryServer.create();
      const memoryUri = mongodInstance.getUri();
      await mongoose.connect(memoryUri);
      console.log(`[Database] Connected successfully to in-memory MongoDB at
${memoryUri}`);
    } catch (memErr) {
      console.error('[Database] Fatal: Failed to initialize in-memory MongoDB:',
memErr.message);
```

```
      process.exit(1);
    }
  }
};

module.exports = { connectDB };
```

```
# ================================================================
# INVOXA Enterprise Platform Configuration
# Note: Sensitive secrets use placeholder values for security
# ================================================================
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173

# Database Connection (MongoDB URI or auto-fallback to embedded
MongoMemoryServer)
MONGODB_URI=mongodb://127.0.0.1:27017/invoiceflow

# Authentication Security (Cryptographic JWT Secret)
JWT_SECRET=[REDACTED_SECURE_JWT_SECRET_KEY_PLACEHOLDER]
JWT_EXPIRES_IN=7d

# File Upload Configuration
MAX_FILE_SIZE_BYTES=5242880
UPLOAD_DIR=./uploads
```

```
// File: server/models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name:       { type: String, required: [true, 'Please provide a name'], trim:
true },
  email:      { type: String, required: [true, 'Please provide an email'],
unique: true, lowercase: true, trim: true },
  password:   { type: String, required: [true, 'Please provide a password'],
minlength: 6, select: false },
  role:       { type: String, enum: ['ADMIN', 'ACCOUNTS TEAM', 'VENDOR'],
default: 'ACCOUNTS TEAM' },
  avatar:     { type: String, default: '' },
  phone:      { type: String, default: '+91 98765 43210' },
  department: { type: String, default: 'Accounts Payable' },
  vendor:     { type: mongoose.Schema.Types.ObjectId, ref: 'Vendor', default:
null },
  isActive:   { type: Boolean, default: true }
}, { timestamps: true });
```

```
// Hash password with bcrypt before saving
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare password helper method
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

```
// File: server/controllers/authController.js (register)
const register = async (req, res) => {
  try {
    const { name, email, password, role, department, vendorId } = req.body;

    // Check if user already exists
    const existingUser = await User.findOne({ email: email.toLowerCase().trim()
});
    if (existingUser) {
      return errorResponse(res, 'User with this email already exists.',
'Conflict', 409);
    }

    // Create user (password automatically hashed by pre-save hook)
    const user = await User.create({
      name,
      email: email.toLowerCase().trim(),
      password,
      role: role || 'ACCOUNTS TEAM',
      department: department || 'Accounts Payable',
      vendor: vendorId || null
    });

    const token = generateToken(user._id, user.role);
    return successResponse(res, { token, user: { id: user._id, name: user.name,
email: user.email, role: user.role } }, 'Registered successfully', 201);
  } catch (err) {
    return errorResponse(res, 'Registration failed.', err.message, 500);
  }
};
```

```
// File: server/controllers/authController.js (login & generateToken)
const generateToken = (id, role) => {
```

```
  return jwt.sign(
    { id, role },
    process.env.JWT_SECRET ||
'invoiceflow_super_secure_jwt_secret_key_2026_xyz',
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
  );
};

const login = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return errorResponse(res, 'Please provide an email and password.',
'Missing credentials', 400);
    }
    const user = await User.findOne({ email: email.toLowerCase().trim()
}).select('+password');
    if (!user || !(await user.matchPassword(password))) {
      return errorResponse(res, 'Invalid email or password.', 'Invalid
credentials', 401);
    }
    if (!user.isActive) {
      return errorResponse(res, 'Your account has been deactivated.', 'Account
deactivated', 403);
    }

    const token = generateToken(user._id, user.role);

    // Record audit activity
    await ActivityLog.create({
      user: user._id, userName: user.name, userRole: user.role,
      action: 'LOGIN', entity: 'AUTH', entityId: user._id.toString(),
      description: `User ${user.name} logged into INVOXA platform successfully`,
      ipAddress: req.ip || '127.0.0.1'
    });

    return successResponse(res, {
      token,
      user: { id: user._id, name: user.name, email: user.email, role: user.role,
department: user.department }
    }, 'Login successful');
  } catch (err) {
    return errorResponse(res, 'Login failed.', err.message, 500);
  }
};
```

```
// File: server/middleware/authMiddleware.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');
```

```
const { errorResponse } = require('../utils/apiResponse');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization &&
req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return errorResponse(res, 'Authentication token required. Please log in.',
'Not authorized', 401);
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET ||
'invoiceflow_super_secure_jwt_secret_key_2026_xyz');
    const user = await User.findById(decoded.id).select('-password');
    if (!user || !user.isActive) {
      return errorResponse(res, 'User inactive or no longer exists.',
'Forbidden', 403);
    }
    req.user = user;
    next();
  } catch (err) {
    return errorResponse(res, 'Invalid or expired token. Please log in again.',
err.message, 401);
  }
};

module.exports = { protect };
```

```
// File: server/controllers/invoiceController.js (createInvoice)
const createInvoice = async (req, res) => {
  try {
    const { invoiceNumber, vendor, invoiceDate, dueDate, lineItems, notes } =
req.body;

    // Calculate subtotal from line items
    const parsedLineItems = typeof lineItems === 'string' ?
JSON.parse(lineItems) : lineItems;
    const subtotal = parsedLineItems.reduce((acc, item) => acc + (item.quantity
* item.unitPrice), 0);
    const taxAmount = Math.round(subtotal * 0.18);
    const totalAmount = subtotal + taxAmount;

    // Run automated 7-point verification and duplicate detection
    const verifyResult = await verifyInvoice({ vendor, invoiceNumber,
invoiceDate, dueDate, subtotal, taxAmount, totalAmount, lineItems:
parsedLineItems });
```

```
    const duplicateResult = await checkDuplicateInvoice({ vendor, invoiceNumber,
totalAmount, invoiceDate });

    const invoice = await Invoice.create({
      invoiceNumber: invoiceNumber.trim().toUpperCase(),
      vendor, invoiceDate, dueDate, subtotal, taxAmount, totalAmount,
      lineItems: parsedLineItems, notes,
      verificationStatus: verifyResult.status,
      verificationScore: verifyResult.score,
      verificationChecks: verifyResult.checks,
      isDuplicate: duplicateResult.isDuplicate,
      duplicateConfidence: duplicateResult.confidence,
      duplicateOf: duplicateResult.duplicateOf ? duplicateResult.duplicateOf._id
: null,
      status: duplicateResult.isDuplicate ? 'FLAGGED_DUPLICATE' :
(verifyResult.status === 'VERIFIED' ? 'PENDING_APPROVAL' : 'NEEDS_REVIEW'),
      createdBy: req.user._id
    });

    return successResponse(res, invoice, 'Invoice created and verified
successfully', 201);
  } catch (err) {
    return errorResponse(res, 'Failed to create invoice', err.message, 500);
  }
};
```

```
// File: server/controllers/invoiceController.js (getInvoices & getInvoiceById)
const getInvoices = async (req, res) => {
  try {
    const { search, status, vendor, page = 1, limit = 10, sortBy = 'createdAt',
sortOrder = 'desc' } = req.query;
    const query = {};

    // Role-based visibility scoping
    if (req.user.role === 'VENDOR') {
      query.vendor = req.user.vendor;
    } else if (vendor) {
      query.vendor = vendor;
    }
    if (status) query.status = status;
    if (search) query.invoiceNumber = { $regex: search.trim(), $options: 'i' };

    const total = await Invoice.countDocuments(query);
    const invoices = await Invoice.find(query)
      .populate('vendor', 'name companyName email phone gstin')
      .populate('createdBy', 'name email')
      .sort({ [sortBy]: sortOrder === 'asc' ? 1 : -1 })
      .skip((page - 1) * limit)
      .limit(parseInt(limit));
```

```
    return successResponse(res, { invoices, total, pages: Math.ceil(total /
limit), currentPage: parseInt(page) });
  } catch (err) {
    return errorResponse(res, 'Failed to retrieve invoices', err.message, 500);
  }
};

const getInvoiceById = async (req, res) => {
  try {
    const invoice = await Invoice.findById(req.params.id)
      .populate('vendor')
      .populate('approvalHistory.actor', 'name role email');
    if (!invoice) return errorResponse(res, 'Invoice not found', '404', 404);
    return successResponse(res, invoice);
  } catch (err) {
    return errorResponse(res, 'Failed to retrieve invoice', err.message, 500);
  }
};
```

```
// File: server/controllers/invoiceController.js (updateInvoice)
const updateInvoice = async (req, res) => {
  try {
    const invoice = await Invoice.findById(req.params.id);
    if (!invoice) return errorResponse(res, 'Invoice not found', '404', 404);

    // Prevent modifications if already locked in payment or completed
    if (['PAID', 'PROCESSING'].includes(invoice.status)) {
      return errorResponse(res, 'Cannot edit an invoice currently being
processed or already paid.', 'Locked', 400);
    }

    const { dueDate, notes, lineItems } = req.body;
    if (dueDate) invoice.dueDate = dueDate;
    if (notes) invoice.notes = notes;
    if (lineItems) {
      invoice.lineItems = lineItems;
      invoice.subtotal = lineItems.reduce((acc, it) => acc + (it.quantity *
it.unitPrice), 0);
      invoice.taxAmount = Math.round(invoice.subtotal * 0.18);
      invoice.totalAmount = invoice.subtotal + invoice.taxAmount;
    }

    const updated = await invoice.save();
    return successResponse(res, updated, 'Invoice updated successfully');
  } catch (err) {
    return errorResponse(res, 'Failed to update invoice', err.message, 500);
  }
```

```
};
```

```
// File: server/controllers/invoiceController.js (deleteInvoice)
const deleteInvoice = async (req, res) => {
  try {
    const invoice = await Invoice.findById(req.params.id);
    if (!invoice) return errorResponse(res, 'Invoice not found', '404', 404);

    if (invoice.status === 'PAID') {
      return errorResponse(res, 'Cannot delete a paid invoice. Archival required
for statutory audit.', 'Prohibited', 400);
    }

    await invoice.deleteOne();

    // Create immutable audit entry
    await ActivityLog.create({
      user: req.user._id, userName: req.user.name, userRole: req.user.role,
      action: 'DELETE', entity: 'INVOICE', entityId: req.params.id,
      description: `Invoice ${invoice.invoiceNumber} permanently removed by
${req.user.name}`
    });

    return successResponse(res, { id: req.params.id }, 'Invoice deleted
successfully');
  } catch (err) {
    return errorResponse(res, 'Failed to delete invoice', err.message, 500);
  }
};
```

```
// File: server/routes/invoiceRoutes.js & server/app.js
const express = require('express');
const router = express.Router();
const { protect } = require('../middleware/authMiddleware');
const { authorize } = require('../middleware/roleMiddleware');
const { getInvoices, getInvoiceById, createInvoice, updateInvoice, deleteInvoice
} = require('../controllers/invoiceController');

// All invoice routes require JWT authentication
router.use(protect);

router.route('/')
  .get(getInvoices)
  .post(authorize('ADMIN', 'ACCOUNTS TEAM', 'VENDOR'), createInvoice);

router.route('/:id')
  .get(getInvoiceById)
```

```
  .put(authorize('ADMIN', 'ACCOUNTS TEAM'), updateInvoice)
  .delete(authorize('ADMIN'), deleteInvoice);

module.exports = router;

// File: server/app.js (Route mounting)
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/invoices', require('./routes/invoiceRoutes'));
app.use('/api/approvals', require('./routes/approvalRoutes'));
app.use('/api/payments', require('./routes/paymentRoutes'));
app.use('/api/vendors', require('./routes/vendorRoutes'));
app.use('/api/reports', require('./routes/reportRoutes'));
app.use('/api/activity', require('./routes/activityRoutes'));
```

```
// File: server/server.js
const dotenv = require('dotenv');
dotenv.config();

const app = require('./app');
const { connectDB } = require('./config/db');
const { runAutomatedReminders } = require('./services/reminderService');

const PORT = process.env.PORT || 5000;

const startServer = async () => {
  try {
    // 1. Connect to Database (auto fallback to embedded MongoMemoryServer if
needed)
    await connectDB();

    // 2. Start HTTP Server
    const server = app.listen(PORT, () => {
      console.log(` INVOXA API Server running on port ${PORT}`);
      console.log(` Environment: ${process.env.NODE_ENV || 'development'}`);
      console.log(` Healthcheck: http://localhost:${PORT}/api/health`);
    });

    // 3. Run initial automated reminder & priority scan after 3 seconds
    setTimeout(() => { runAutomatedReminders(); }, 3000);

    // 4. Schedule periodic reminder sweeps every 30 minutes
    setInterval(runAutomatedReminders, 30 * 60 * 1000);

    // Graceful Shutdown
    process.on('SIGTERM', () => {
      console.log('SIGTERM received. Shutting down gracefully...');
      server.close(() => console.log('Server terminated.'));
    });
```

```
  } catch (err) {
    console.error('Fatal: Failed to start server:', err);
    process.exit(1);
  }
};

startServer();
```

```
// File: client/src/pages/auth/LoginPage.jsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import { useToast } from '../../context/ToastContext';
import { Eye, EyeOff, Lock, Mail } from 'lucide-react';

export const LoginPage = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [loading, setLoading] = useState(false);
  const [errorMessage, setErrorMessage] = useState('');

  const { login } = useAuth();
  const { showToast } = useToast();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!email || !password) {
      setErrorMessage('Please enter both email and password.');
      return;
    }
    setLoading(true);
    setErrorMessage('');
    const res = await login(email, password);
    if (res.success) {
      showToast(`Welcome back, ${res.user.name}!`, 'success');
      navigate('/dashboard');
    } else {
      setLoading(false);
      setErrorMessage('Invalid email or password. Please verify your
credentials.');
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-[#1A1817]
px-4">
```

```
      <div className="max-w-md w-full bg-[#242120] rounded-xl p-8 border border-
[#332F2C] shadow-2xl">
        <h2 className="text-2xl font-bold text-white text-center mb-6">Sign In
to INVOXA</h2>
        {errorMessage && <div className="mb-4 p-3 bg-red-900/30 text-red-400
text-sm rounded-lg">{errorMessage}</div>}
        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="text-xs font-semibold text-gray-400">Email
Address</label>
            <input type="email" value={email} onChange={(e) =>
setEmail(e.target.value)}
              className="w-full mt-1 px-4 py-2.5 bg-[#1A1817] text-white border
border-[#332F2C] rounded-lg" required />
          </div>
          <div>
            <label className="text-xs font-semibold text-gray-
400">Password</label>
            <input type={showPassword ? 'text' : 'password'} value={password}
onChange={(e) => setPassword(e.target.value)}
              className="w-full mt-1 px-4 py-2.5 bg-[#1A1817] text-white border
border-[#332F2C] rounded-lg" required />
          </div>
          <button type="submit" disabled={loading}
            className="w-full py-3 bg-[#E05A47] hover:bg-[#D04936] text-white
font-bold rounded-lg transition">
            {loading ? 'Authenticating...' : 'Sign In'}
          </button>
        </form>
      </div>
    </div>
  );
};
```

```
// File: client/src/pages/dashboard/DashboardPage.jsx
import React, { useState, useEffect, useCallback } from 'react';
import { api } from '../../services/api';
import StatCard from '../../components/common/StatCard';
import OverdueBanner from '../../components/dashboard/OverdueBanner';

export const DashboardPage = () => {
  const [metrics, setMetrics] = useState(null);
  const [loading, setLoading] = useState(true);

  const fetchDashboardData = useCallback(async () => {
    try {
      // Parallel execution of financial metrics, activity feed, and pending
approval queues
      const [metricRes, actRes, appRes] = await Promise.all([
        api.get('/api/reports/dashboard'),
```

```
        api.get('/api/activity?limit=5').catch(() => ({ data: { logs: [] } })),
        api.get('/api/approvals').catch(() => ({ data: [] })),
      ]);
      if (metricRes.success && metricRes.data) {
        setMetrics(metricRes.data);
      }
    } catch (err) {
      console.error('Failed to load dashboard statistics', err);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => { fetchDashboardData(); }, [fetchDashboardData]);

  return (
    <div className="space-y-6">
      {metrics?.overdueList?.length > 0 && (
        <OverdueBanner invoice={metrics.overdueList[0]} />
      )}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <StatCard title="TOTAL INVOICES" value={metrics?.summary?.totalInvoices
|| 0} />
        <StatCard title="PENDING VERIFICATION"
value={metrics?.summary?.pendingVerification || 0} />
        <StatCard title="PENDING APPROVAL"
value={metrics?.summary?.pendingApproval || 0} />
        <StatCard title="OVERDUE LIABILITIES" value={`₹` +
(metrics?.summary?.overdueAmount || 0).toLocaleString()} />
      </div>
    </div>
  );
};
```

ADD CODE - Request

```
// ---------------------------------------------------------------
// EXAMPLE 1: User Authentication & Token Generation
// ---------------------------------------------------------------
// POST /api/auth/login
REQUEST PAYLOAD:
{
  "email": "admin@example.com",
  "password": "Admin@123"
}

RESPONSE PAYLOAD (HTTP 200 OK):
{
  "success": true,
```

```
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1ZjEx...",
    "user": {
      "id": "6ace51fbf53933636df8a101",
      "name": "Yuva Umaya Shri A",
      "email": "admin@example.com",
      "role": "ADMIN",
      "department": "Finance & Operations"
    }
  }
}

// ------------------------------------------------------------
// EXAMPLE 2: Executive Financial Dashboard KPI Query
// ------------------------------------------------------------
// GET /api/reports/dashboard (Headers: Authorization: Bearer <TOKEN>)
RESPONSE PAYLOAD (HTTP 200 OK):
{
  "success": true,
  "data": {
    "summary": {
      "totalInvoices": 21,
      "pendingVerification": 2,
      "pendingApproval": 6,
      "paymentsDueSoon": 4,
      "paymentsDueSoonAmount": 212900,
      "overduePayments": 1,
      "overdueAmount": 124000,
      "completedPayments": 7,
      "totalPayableAmount": 937400,
      "totalPaidAmount": 676000
    },
    "overdueList": [
      {
        "id": "6ace51fbf53933636df8a105",
        "invoiceNumber": "INV-2026-002",
        "vendorName": "CloudNova Solutions",
        "amount": 124000,
        "dueDate": "2026-10-03T15:44:59.192Z",
        "daysOverdue": 11,
        "status": "OVERDUE",
        "paymentId": "6ace51fbf53933636df8a108"
      }
    ]
  }
}
```

SCREENSHOTS OF THE PROJECT

Figure A.1: User Login and Multi-Role Authentication Screen (JWT Bearer Security)

Figure A.2: Executive Action Center Dashboard (4 KPI Summary Cards, Urgent Overdue Liability Banner, and Real-Time Priority Queue)

Figure A.3: Automated Invoice 7-Point Integrity Verification and Multi-Signal Duplicate Detection Audit Screen

Figure A.4: 4-Stage Enterprise Payment Settlement Simulation with Bank UTR Reference Number Generation

[10] Meta Platforms Inc., 'React 18 Official Documentation: Concurrent Rendering, Hooks, and Virtual DOM Architecture,' 2026. [Online]. Available: https://react.dev/